

CSCE 711 APPLIED CRYPTOGRAPHY · MODULE 11 · LECTURE NOTES

Implementation Hazards

How correct designs fail in code.

MODULE OUTCOMES

- Explain timing side channels and constant-time programming.
- Explain nonce reuse consequences per construction.
- Explain what other side channels exist.
- Explain why error handling is a cryptographic concern.
- Review cryptographic code for these hazards.

1 Timing

IF EXECUTION TIME DEPENDS ON SECRET DATA, THE SECRET LEAKS

Any conditional branch on a secret value, any memory access at a secret-dependent index, and any loop whose iteration count depends on a secret, leaks information through timing. The mathematics being correct is irrelevant to this.

And the leak is exploitable remotely, not merely in theory. Network jitter averages out over sufficiently many samples, and cache-timing attacks have recovered full keys across process boundaries and across virtual machine boundaries on shared hardware — which is the ordinary deployment model for cloud computing.

So constant-time means precisely this: the same operations, in the same order, touching the same memory, regardless of the secret value — no early exit, no secret-indexed table lookup, no conditional branch on secret data.

Which is genuinely hard to write and considerably harder to preserve — a compiler may reintroduce a branch while optimising code that was constant-time in its source form, because the compiler is optimising for speed and knows nothing about your threat model. This is the single strongest argument for Module 01 §4's rule: the people who maintain libsodium check the generated assembly, and you will not.

```

WRONG                                RIGHT
if secret: a() else b()              compute both, select with a mask
memcmp(tag, expected)                constant-time compare
table[secret_byte]                   bitsliced or table-free
                                     implementation
while x: ...                          a fixed iteration count
return early on error                 one exit, after all the work

```

```

SELECTION WITHOUT BRANCHING
    mask = -(condition)              // 0 or all ones
    result = (a & mask) | (b & ~mask)

```

AND VERIFY IT HOLDS IN THE BINARY, not in the source.
Compilers reintroduce branches; a constant-time source file is not a constant-time program. Use `dudect`, `ctgrind`, or your platform's checker, and read the generated assembly for the hot path.

Verify in the binary — the most common failure in practice is a correctly written constant-time source file that the compiler optimised into a branch, which no amount of source review will catch. This is what distinguishes real practice from the folklore, and it is why constant-time primitives are frequently written in assembly or in a restricted dialect with optimisation barriers.

2 Nonce reuse

Construction	The consequence of reusing a nonce
CTR mode / any stream cipher	The two plaintexts XORed together, and both usually recoverable (Module 03 §3's equation).
AES-GCM	All of the above, <i>plus</i> the authentication key becomes recoverable — after which the adversary can forge valid ciphertexts at will. See the note.
CBC (IV reuse)	Reveals whether two messages share a common prefix, and how long it is — less catastrophic, and still a leak.
AES-GCM-SIV	Reveals only whether the two plaintexts were identical — which is the point of a misuse-resistant construction.
ECDSA (nonce reuse or bias)	The private signing key is recoverable from two signatures (Module 07 §1) — total compromise.

GCM is the worst case, and it is also the most widely deployed. A repeated nonce costs you confidentiality *and* the ability to detect forgery, because the authentication key is derived in a way that two ciphertexts under one nonce expose — so the adversary moves from reading to writing. **Which is exactly why AES-GCM-SIV and XChaCha20 exist, and ranking the consequences this way is what makes the misuse-resistant choice obvious** rather than merely advisable.

UNIQUENESS IS HARDER THAN IT SOUNDS IN A REAL SYSTEM

A counter is unique until the process restarts and loses it, or until a virtual machine snapshot is restored to an earlier state, or until the service is scaled horizontally and two instances hold the same counter — each of which has caused real nonce reuse.

And a random nonce is unique only probabilistically. At 96 bits — which is AES-GCM's nonce size — the birthday bound makes a collision realistic after a few billion messages under a single key, which large systems reach.

So the practical answers are structural: a 192-bit random nonce (XChaCha20-Poly1305), a misuse-resistant construction (AES-GCM-SIV), or a fresh key per message via envelope encryption (Module 10 §2) — any of which removes the requirement rather than managing it.

Which makes this a design choice rather than a discipline problem — and choosing a construction that cannot fail this way is strictly better than operating a procedure that must not, because procedures are executed by systems under conditions their designers did not anticipate.

3 The other channels

- **Cache timing** — which is §1, and is **the one that genuinely matters for ordinary server software** running on shared hardware.
- **Power analysis and electromagnetic emission** — **relevant for smartcards, hardware tokens, and any device you hand to an adversary**, and generally not relevant for a server in a datacentre you control.
- **Fault injection** — deliberately inducing computational errors with voltage or clock glitching to extract keys, which again requires physical possession and is a hardware design concern.
- **Speculative execution** — which crossed isolation boundaries previously believed solid, and is mitigated at the platform and compiler level rather than by application code (CSCE 701 Module 05).
- **And compression ratio** — **a logical rather than a physical channel, and precisely the reason TLS removed compression** (Module 08 §2). **Be honest about your threat model here: power analysis is a real and serious attack, and it is almost certainly not *your* attack unless you ship hardware** — and spending effort on it while leaving a memcmp on a tag is the wrong allocation.

4 Errors, and review

DISTINGUISHABLE ERRORS BUILD A DECRYPTION ORACLE

If 'bad padding', 'bad authentication tag', and 'malformed length' are distinguishable from one another — by error message, by HTTP status code, by response size, or by response timing — then an adversary can use your system as a decryption service.

Which is exactly how padding oracle attacks recover plaintext without ever obtaining the key, using nothing but the server's own error behaviour as the oracle (Module 04 §4) — and the attack is efficient, requiring a few hundred queries per byte.

So: one opaque error for every cryptographic failure, returned after performing the same amount of work in every case, and **with the diagnostic detail logged internally where the adversary cannot read it** (CSCE 701 Module 09 §1).

And this is the hazard most likely to appear in code you personally wrote — the primitives are inside a library maintained by specialists, and the error handling is in your own request handler, written while thinking about debuggability rather than about oracles. **Which makes it the highest-yield thing to look for in your own code.**

- **Any == or memcmp applied to a tag, a token, a hash, or a signature** — **the single most findable defect in this module**, and a grep away.
- **Any nonce or IV supplied by application code**, and specifically where that value comes from and whether its uniqueness survives a restart or a scale-out (§2).
- **Any distinguishable error path** following a decryption or verification failure — including differences in logging, timing, and response shape rather than only in the message text.

- **Any key used for two purposes**, or any key derived without a per-purpose context label (Module 09 §3) — which is cheap to fix and easy to miss.
- **And anything that parses, decompresses, deserialises, or logs before verifying** — which is Module 04 §4's ordering rule, and is where cryptographic failure meets CSCE 713's memory-safety and injection material. **These five checks find most real defects**, and **all five are mechanical enough to automate at least partially in a linter**, which is a worthwhile afternoon.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Aumasson — Serious Cryptography , the implementation chapters	§1 and §2 at engineering level , with the constant-time patterns and the misuse consequences.
BearSSL — Constant-Time Cryptography (free)	§1's best single reference — thorough, practical, and honest about what the compiler does to you.
Bernstein — Cache-timing attacks on AES (free)	§1's threat demonstrated — and the paper that changed how AES is implemented in software.
Joux — Authentication failures in NIST GCM (free)	§2's GCM row, in the original — why nonce reuse costs the authentication key specifically.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write a non-constant-time tag comparison** and measure the timing difference across guesses.
- 2 **Implement branchless selection with a mask** and verify it produces the right answer.
- 3 **Compile a constant-time function with optimisation on** and read the generated assembly.
- 4 **Find whether the compiler reintroduced a branch**, and report what you found.
- 5 **Reuse a nonce with AES-GCM** on your own test data and recover the two plaintexts.
- 6 **Do the same with AES-GCM-SIV** and compare what you can extract.
- 7 **Determine whether your nonce scheme survives** a process restart and a horizontal scale-out.
- 8 **Switch one construction to XChaCha20** and note what requirement disappeared.
- 9 **Audit one error path** for distinguishability in message, status, size, and timing.
- 10 **Run Part 4's five checks** across a codebase and report the findings.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the timing principle and the three things that leak.
- 2 Why is a timing leak remotely exploitable?
- 3 Give five wrong patterns and their constant-time replacements.
- 4 Why must you verify in the binary?
- 5 Give the nonce reuse consequence for five constructions.
- 6 Why is GCM the worst case?
- 7 Why is uniqueness hard, and what are the three structural answers?
- 8 Name five other side channels and say which matters to you.
- 9 How does a distinguishable error build an oracle?
- 10 Give the five review checks.